TUPLES & SETS

Python's Special Collections

( )

{ }

[ ]

Grade Level:	Grade 4 (Ages 9–10)
Subject:	Python Programming
Duration:	45–60 minutes
Prerequisites:	Basic Python variables & lists

■ Learning Objectives

1. Understand what tuples and sets are in Python
2. Know the key differences: ordered vs unordered, mutable vs immutable
3. Create, access, and modify tuples and sets
4. Decide when to use a tuple or a set
5. Practise coding with relatable Indian examples

1 What Are Collections in Python?

In Python, a **collection** is a way to group many pieces of data together under one name. Instead of creating a separate variable for every item, you store them all in one place and work with them efficiently.

Python has three main built-in collection types that you will learn about in this lesson:

Collection	Symbol	Key Idea
List	[ ]	Ordered, changeable, allows duplicates
Tuple	( )	Ordered, **unchangeable**, allows duplicates
Set	{ }	Unordered, changeable, **NO duplicates**

■

Think of collections like different kinds of containers. A LIST is like a shopping bag — you can rearrange and add items. A TUPLE is like a sealed package — the contents are fixed. A SET is like a unique stamp collection — no two stamps are the same!

2 Tuples — Ordered & Unchangeable

2.1 What Is a Tuple?

A **tuple** is a collection that stores items **in a fixed order**. Once you create a tuple, the items and their positions **cannot be changed**. Tuples are called **immutable** — meaning they cannot be mutated (changed).

2.2 The Rajdhani Express Analogy ■

Picture the Rajdhani Express train. It has coaches in a fixed sequence: S1, S2, S3, AC1, AC2. You cannot swap or rearrange the coaches once the train has been set — they are permanent. A tuple works exactly the same way!

Real World	Tuple
Rajdhani coaches (S1, S2, S3 …)	Items in a tuple
Coaches have fixed positions	Items have fixed index numbers
Cannot rearrange coaches	Cannot change tuple items
Aadhaar name/DOB stays same	Tuple data stays the same

2.3 Creating a Tuple

Use ( ) — parentheses — and separate items with commas:

```
# Creating a tuple
train_coaches = ("S1", "S2", "S3", "AC1", "AC2")

# Print the whole tuple
print(train_coaches)
# Output: ("S1", "S2", "S3", "AC1", "AC2")

# Count items
print(len(train_coaches))   # Output: 5
```

2.4 Accessing Tuple Items (Indexing)

Each item in a tuple has an **index number** that shows its position. Indexing in Python always starts at **0**, not 1.

S1	S2	S3	AC1	AC2
0	1	2	3	4

Index positions of each coach in the tuple

```
print(train_coaches[0])   # Output: S1   (first item)
print(train_coaches[3])   # Output: AC1  (fourth item)
print(train_coaches[-1])  # Output: AC2  (last item)
```

2.5 Tuple Properties

Ordered: Items always stay in the same sequence.

Immutable: Items cannot be added, removed, or changed after creation.

Duplicates Allowed: The same value can appear more than once.

Mixed Types OK: A tuple can hold strings, numbers, and other types together.

2.6 What Happens When You Try to Change a Tuple?

```
train_coaches[0] = 'B1'   # Trying to change S1 to B1

# ERROR OUTPUT:
# TypeError: 'tuple' object does not support item assignment
```

Tuples are IMMUTABLE — once created, you cannot change, add, or remove any item. Python will throw a TypeError if you try. This is a feature, not a bug — it keeps your data safe!

2.7 When Should You Use a Tuple?

- Days of the week — they never change (Monday, Tuesday ...)
- Your date of birth — always fixed
- Coordinates of a location — (latitude, longitude)
- Fixed IPL teams for a season
- Indian states listed in a specific order
- Any data that must stay constant and safe from accidental changes

3 Sets — Unique & Unordered

3.1 What Is a Set?

A **set** is a collection that stores **only unique items** with **no fixed order**. If you add a duplicate item to a set, Python silently removes it — the set keeps only one copy. Sets are **mutable**, meaning you can add or remove items after creation.

3.2 The Dabba (Tiffin Box) Analogy ■

Imagine your school dabba (tiffin box) contains rajma, chana, and moong dal. If you accidentally add rajma again, the duplicate disappears automatically — your dabba always has only unique dals. That's exactly how a Python set works!

Real World	Set
Different dals in a dabba	Items in a set
No dal repeats in the dabba	No duplicate items allowed
Can swap the dabba's contents	Can add or remove items (mutable)
Roll call register — unique names only	Set stores only unique values

3.3 Creating a Set

Use { } — curly braces — and separate items with commas:

```
# Creating a set (note: rajma appears twice!)
dals = {"rajma", "chana", "moong", "rajma"}

print(dals)
# Output: {'rajma', 'chana', 'moong'}   <-- only ONE rajma!

print(len(dals))   # Output: 3
```

■

Notice that the duplicate 'rajma' was removed automatically. Python sets always keep only unique values. The order you see in the output may also be different every time you run the program — that is normal because sets are unordered!

3.4 Set Properties

Unordered: Items have no fixed position — you cannot predict the print order.

Mutable: You can add or remove items after the set is created.

No Duplicates: Each item can appear only once. Duplicates are removed automatically.

No Indexing: You cannot use set[0] — sets have no index numbers.

3.5 Adding and Removing Items

Even though sets have no index, you can still change their contents using built-in methods:

```
student_names = {"Priya", "Arjun", "Sneha"}

# Add a new student
student_names.add("Kiran")
print(student_names)   # {'Priya', 'Arjun', 'Sneha', 'Kiran'}

# Remove a student (safe — no error if item missing)
student_names.discard("Sneha")
print(student_names)   # {'Priya', 'Arjun', 'Kiran'}

# Remove (raises KeyError if item not found)
student_names.remove("Arjun")
```

3.6 Checking Membership with 'in'

The `in` operator checks whether an item exists in a set (very fast!):

```
my_foods = {"idli", "dosa", "biryani", "paratha"}

print("biryani" in my_foods)   # True
print("pizza"   in my_foods)   # False
```

3.7 What Happens If You Try to Index a Set?

```
student_names[0]   # Trying to use index 0

# ERROR OUTPUT:
# TypeError: 'set' object is not subscriptable
```

■■

Sets CANNOT be indexed. There is no 'first' or 'second' item in a set because sets are unordered. If you need to access items by position, use a list or tuple instead.

3.8 When Should You Use a Set?

- Tracking unique states visited on a trip
- Languages spoken in your class (no duplicates needed)

- Items available in the school canteen
- Sports offered at school — each sport listed once
- Removing duplicates quickly from a large list
- Fast membership testing (is X in this group?)

4 Tuple vs Set vs List — Side-by-Side

Feature	List [ ]	Tuple ( )	Set { }
Ordered?	✓ Yes	✓ Yes	✗ No
Can Change?	✓ Yes	✗ No	✓ Yes
Duplicates?	✓ Yes	✓ Yes	✗ No
Indexing?	✓ Yes	✓ Yes	✗ No
Speed for lookup?	Slower (O n)	Slower (O n)	Very Fast (O 1)
Use case example	Shopping list	Train coaches, DOB	Unique class names

5 Coding Activity — My Indian Data Box ■

Now it's your turn! Follow the steps below to create your own tuple of Indian states and a set of favourite Indian foods. Try every step and observe what Python does.

Step 1 — Create Your State Tuple

```
my_states = ("Maharashtra", "Kerala", "Rajasthan", "Punjab", "Goa")
print(my_states)
print("First state:", my_states[0])
print("Last state:", my_states[-1])
print("Total states:", len(my_states))
```

Step 2 — Try to Change It (Observe the Error!)

```
my_states[0] = "Gujarat"   # This will cause a TypeError!
# Expected output:
# TypeError: 'tuple' object does not support item assignment
```

Step 3 — Create Your Food Set

```
my_foods = {"idli", "dosa", "paratha", "biryani", "idli"}
print(my_foods)            # idli appears only ONCE
print("Unique foods:", len(my_foods))
```

Step 4 — Add and Remove Items

```
my_foods.add("pani puri")   # Add a new item
print(my_foods)
my_foods.discard("dosa")    # Remove an item
print(my_foods)
```

Step 5 — Check Membership

```
print("biryani" in my_foods)   # True
print("pizza" in my_foods)     # False
print("paratha" in my_foods)   # ?
```

Challenge: Create a tuple of the 7 days of the week and a set of subjects you study at school. Try printing the third day and checking if 'Maths' is in your subjects set!

6 Key Vocabulary

Tuple	A collection that is ordered and immutable (cannot be changed). Uses ( ) parentheses.
Set	A collection with only unique items and no fixed order. Uses { } curly braces.
Immutable	Something that cannot be changed after it is created. Tuples are immutable.
Mutable	Something that CAN be changed after creation. Lists and sets are mutable.
Ordered	Items have a fixed position (first, second, third…). Lists and tuples are ordered.
Unordered	Items have no fixed position and can appear in any sequence. Sets are unordered.
Index	The position number of an item, starting from 0. Used in lists and tuples.
Duplicate	The same item appearing more than once. Sets automatically remove duplicates.

TypeError	An error Python raises when you try an operation on the wrong type, e.g. indexing a set.

7 Lesson Summary

Tuples ( )	Sets { }
Ordered — items have fixed positions	Unordered — no fixed positions
Immutable — cannot be changed after creation	Mutable — can add or remove items
Duplicates allowed	No duplicates — only unique items
Indexing supported: tuple[0]	Indexing NOT supported
Like a train with fixed coaches	Like a dabba with unique dals

8 Quick Quiz — Test Your Knowledge!

Q1. What symbol do you use to create a tuple?

a) [ ] b) { } c) ( ) d) < >

Q2. What will happen if you run: my_tuple[0] = 'new'?

a) The item changes b) TypeError c) Nothing d) SyntaxError

Q3. You create: s = {1, 2, 3, 2, 1}. What is len(s)?

a) 5 b) 3 c) 2 d) 6

Q4. Which collection is best for storing your date of birth?

a) Set b) List c) Tuple d) Dictionary

Q5. Which method adds an item to a set?

a) .append() b) .insert() c) .add() d) .push()

Q6. Can you do set_name[0] to get the first item of a set?

a) Yes b) No, sets are unordered c) Only for numbers d) Yes, with -1

Answers: Q1-c, Q2-b, Q3-b, Q4-c, Q5-c, Q6-b

9 Teacher / Parent Notes

- This lesson is designed for students who have already been introduced to Python variables and basic lists.
- The Indian analogies (Rajdhani train, dabba with dals) make abstract concepts tangible for Grade 4 students.
- Allow students to run the TypeError code intentionally — seeing errors in a safe environment builds debugging confidence.
- Extension activity: Ask students to convert a list with duplicates into a set and count how many duplicates were removed.
- For advanced students: Introduce frozenset as an immutable version of a set.
- Recommended tools: Python IDLE, Thonny IDE, or an online playground like replit.com or trinket.io.